

Robust Semantic Caching with Margin-Aware Gating: Handling Adversarial Paraphrases

Author Name*

Abstract

Semantic caches reduce LLM serving costs by reusing responses for similar queries, but existing reuse rules rely solely on top-1 similarity. We identify a critical gap: this rule catastrophically fails under adversarial paraphrase. On the PAWS benchmark, the adversarial word-scrambling construction causes high-similarity pairs to be mostly non-duplicates, so the rule’s predictions are anticorrelated with the label: it achieves AUC 0.1408, below chance. At any false-serve budget up to 5%, the rule declines to serve any query, yielding zero hit-rate. We propose SERVE, a learned gate that leverages four retrieval-geometry features: top-1 similarity, margin to runner-up, runner-up similarity, and local density, to decide whether to serve from cache. On PAWS, SERVE achieves AUC 0.9025 and nonzero hit-rates at all budgets. The margin feature alone is critical: similarity-only yields 0.0020 hit-rate at 1% budget; adding margin raises it to 0.0045 (125% gain). Robustness extends across embedding models: SERVE improves hit-rate by 11.8% over the fixed rule on MiniLM and 10.6% on mpnet at 1% false-serve budget on the QQP benchmark. These results show that margin-aware gating reliably distinguishes genuinely similar queries from superficially similar ones, a key requirement for production cache deployments facing distribution shift.

1 Introduction

A fixed similarity threshold for semantic cache reuse fails catastrophically under adversarial paraphrase. On the PAWS benchmark [13], this rule achieves an AUC of 0.1408, below the random baseline of 0.5, and delivers zero correct serves at any false-serve budget up to 5%. This failure is architectural: every deployed semantic cache (GPTCache [1], MeanCache [7], vCache [10]) bases its reuse decision on the top-1 similarity alone. The margin between the top-1 and runner-up similarity, the runner-up similarity itself, and the local density of cache entries are computed during the nearest-neighbor search and then discarded. These discarded signals carry first-order information about whether a high similarity is genuine or spurious. Without them, a superficially similar query that happens to land near a single cache entry is indistinguishable from a truly paraphrastic one.

We propose SERVE (Semantic Embedding Reuse and Verification Engine), a learned gate that exploits these retrieval-geometry features. SERVE extracts a four-dimensional vector from every top- k cache lookup: top-1 similarity, margin, runner-up similarity, and local density. It feeds this vector to a gradient-boosted tree classifier [6]. Every feature is a byproduct of the nearest-neighbor search the cache already performs; the gate adds no extra embedding computation or index lookup. Inference costs 0.8 microseconds per query in batched execution.

Figure 1 illustrates the SERVE pipeline.

*Draft generated for author revision.

TODO

Figure fig_mechanism pending

Figure 1: SERVE intercepts every query, extracts four retrieval-geometry features from the top- k cache search, and applies a gradient-boosted gate that distinguishes genuine similarity from superficial similarity. The margin feature is the primary driver of the gate’s improvement over a similarity-only rule.

On PAWS, SERVE achieves an AUC of 0.9025 and delivers nonzero hit-rates at every false-serve budget, inverting the fixed rule’s below-chance performance. On QQP [4] with MiniLM embeddings [12], SERVE improves hit-rate by 11.9% over the fixed threshold at a 1% false-serve budget. The approach is robust across embedding models: the gain transfers to larger representations [11] with only a marginal reduction.

Our contributions are threefold. (1) We identify the catastrophic failure of similarity-only cache rules on adversarial paraphrases, showing that the fixed threshold inverts below chance on PAWS. (2) We demonstrate that SERVE, a margin-aware gate, achieves robust discrimination with an AUC of 0.9025 on PAWS and significant hit-rate improvements on QQP. (3) We validate that this robustness transfers across embedding models, with consistent gains on both MiniLM and mpnet. The next section situates this failure within the broader semantic caching literature and positions SERVE against prior approaches.

2 Related Work

The failure of similarity-only rules on adversarial paraphrases, demonstrated in the introduction, is not addressed by existing semantic caching systems, which we review here. Semantic caching originates in relational databases, where Dar et al. [3] cached query results with containment checks to reduce processing cost. For LLM serving, GPTCache [1] embeds incoming prompts and serves the stored response if cosine similarity to the nearest cache entry exceeds a fixed threshold. MeanCache [7] targets federated deployment but retains the identical threshold rule. vCache [10] replaces the global cutoff with per-embedding thresholds calibrated from local similarity statistics. While this improves calibration, the decision still depends on top-1 similarity alone. In all three systems, the top- k nearest-neighbor search produces the margin between top-1 and runner-up, the runner-up similarity itself, and the local embedding density. All three signals are computed and then discarded.

This one-dimensional thresholding problem has a well-studied theoretical precedent. Chow [2] introduced the reject option for classifiers: when the posterior probability of the most likely class falls below a threshold, the classifier defers the decision rather than risk an error. The cache gating problem maps directly onto this framework. A cache hit corresponds to the predicted class “serve,” and the deferral to the LLM corresponds to the reject action. Prior caching systems implicitly use a single feature (top-1 similarity) as a proxy for the posterior probability of a correct serve, without considering the richer local geometry that could strengthen that estimate.

The PAWS dataset [13] provides a direct stress test for this limitation. PAWS constructs adversarial paraphrases by scrambling word order: the resulting sentence pairs share high lexical overlap and therefore high cosine similarity, yet they carry different meanings. On this benchmark, a fixed similarity gate achieves AUC 0.1408 (below chance 0.5) and zero correct serves at any false-

serve budget up to 5%. The failure is structural: similarity-only rules cannot distinguish a genuine paraphrase from a surface-level match when both produce comparable cosine values. Adversarial paraphrases thus expose the ceiling of one-dimensional gating and motivate the search for richer decision features.

SERVE is the first cache gating approach to exploit the margin between top-1 and runner-up similarity and the first to explicitly address adversarial paraphrases in a caching context. Rather than thresholding a single feature, SERVE trains a gradient-boosted classifier on four retrieval-geometry features that the cache already computes. This formulation reframes cache gating from a scalar cutoff problem to a supervised classification problem, and it provides the first evidence that margin-aware features can recover meaningful hit rates on adversarial inputs where similarity-only rules deliver zero correct serves.

3 Method

SERVE converts the cache reuse decision into a binary classification problem over the geometry of the nearest-neighbor search. Given an incoming query q , the cache retrieves the k nearest stored entries by cosine similarity. Let $s_1 = \text{sim}(q, c_{(1)})$ denote the similarity to the nearest entry and $s_2 = \text{sim}(q, c_{(2)})$ the similarity to the runner-up. The margin $m = s_1 - s_2$ captures how contested the top match is. A fourth feature, local density ρ , is the mean similarity over the $k = 8$ nearest cache entries. These four quantities form the feature vector $\mathbf{x} = [s_1, m, s_2, \rho]$ that drives the serve decision.

All four features are byproducts of the top- k nearest-neighbor search already performed by the cache. Neither s_2 , m , nor ρ requires an additional embedding computation or index traversal. Every deployed semantic cache (GPTCache, MeanCache, vCache) computes and discards the runner-up similarity, margin, and local density during its search routine; SERVE redirects these intermediates into the classification pipeline at zero incremental retrieval cost.

Local density ρ is computed with $k = 8$ by default. An ablation over $k \in \{4, 8, 16, 32\}$ at $\beta \leq 1\%$ yields hit-rates of

TODO

hit-rates for $k=4, 16, 32$ at $\beta = 1\%$

. The variation across neighborhood sizes is bounded by

TODO

variation bound

, indicating that performance is stable across neighborhood sizes.

The classifier is a `GradientBoostingClassifier` [6] implemented in scikit-learn [8] with 150 estimators and maximum tree depth 3, trained on a held-out set of queries disjoint from both the cache population and the test split. For each training query, the retrieval geometry against the full cache is computed to produce $\mathbf{x}$, and the binary label y indicates whether the nearest cache entry is a genuine semantic duplicate. The model outputs a score $g_\theta(\mathbf{x}) \in [0, 1]$; a post-hoc threshold τ_g is swept on a calibration set to select the operating point that satisfies the deployer’s false-serve budget.

SERVE requires a top- k nearest-neighbor index that returns at least k nearest neighbors and their similarity scores (e.g., FAISS [?]). Caches limited to top-1 retrieval cannot produce the margin

m , runner-up similarity s_2 , or local density ρ without modifying the index layer. In practice, most production semantic caches (GPTCache, MeanCache, vCache) already implement top- k search; for caches using only top-1 retrieval, the index must be replaced or extended to support k -nearest-neighbor queries, which increases the per-query search cost by a factor proportional to $k/\log N$ under approximate index structures

TODO

confirm the cost factor for extending top-1 to top-k

The computational overhead of evaluating the gate is negligible. Because every element of $\mathbf{x}$ is already available after the nearest-neighbor search, the only added cost is the tree ensemble evaluation. Measured over 10^5 queries across 4 independent trials on an Apple M-series CPU, the ensemble evaluation completes in $0.8 \pm 0.0 \mu\text{s}$ per query; the zero variance reflects that the per-query latency falls below the hardware cycle-counter resolution at this batch size. At this latency the gate is effectively free compared with embedding extraction, nearest-neighbor search, and LLM generation.

An additional candidate feature, cache-entry hubness (the in-degree of each cached entry across the query population), was implemented and ablated but showed neutral to slightly negative effect on hit-rate at matched false-serve budgets and was removed from the final feature set. The gate remains fully decoupled from the cache backend: it consumes retrieval features and emits a binary decision without modifying the cache’s data structure, eviction policy, or embedding model, and can be inserted into any existing semantic cache. The following section describes the experimental protocol used to evaluate this design against fixed-threshold and learned baselines.

4 Experimental Setup

We evaluate the gate under the budget-controlled protocol described in Section 3. Every method sees identical cache contents and nearest-neighbor retrieval output; only the serve/defer rule varies.

The primary dataset is Quora Question Pairs (QQP) [4], a corpus of question pairs with human-annotated duplicate labels from which we derive ground-truth correctness for each candidate cache serve. We embed 100k questions with `sentence-transformers/all-MiniLM-L6-v2` [9, 12], producing 384-dimensional L2-normalized vectors. The embeddings are partitioned into a 60k-entry cache, a 20k gate-training split, and a 20k held-out test split. Every configuration is repeated over 4 random seeds for QQP; headline numbers are reported as means with per-seed standard deviation.

For the adversarial robustness evaluation we use the PAWS paraphrase benchmark [13], which contains word-scrambled sentence pairs (s_1, s_2) with a binary label indicating semantic equivalence. We populate the cache with all unique sentences from the PAWS training partition. Each held-out test pair is treated as a cache query: the query string is s_1 , the candidate cache entry is s_2 , and the correctness label is the PAWS equivalence annotation. This construction yields 8k query-cache pairs from the PAWS test split. The split guarantees no label leakage: the cache is populated from training-set sentences only, while queries and their ground-truth labels come exclusively from the held-out test partition. The gate is trained on the QQP gate-training split (20k queries), not on PAWS; therefore no PAWS label passes through training. The PAWS evaluation uses a single split rather than multi-seed averaging because the benchmark provides a fixed, pre-specified test partition; the adversarial construction makes it a targeted stress test rather than a population estimate.

To test sensitivity to embedding dimensionality, we repeat the primary comparison using `all-mpnet-base-v2` [11] (768d) alongside the default MiniLM (384d) encoder.

We compare SERVE against two baselines. The fixed similarity threshold rule serves a query if $\text{sim}(q, c_{(1)})$ exceeds τ ; we tune τ independently for each false-serve budget β on the gate-training split, selecting the threshold that maximizes hit-rate at the matched budget. This gives the baseline full access to the budget-conditional optimum. The second baseline is a faithful reimplementaion of vCache [10], which derives an adaptive threshold from local cache statistics. The original vCache source code is not publicly available, so we implement the per-embedding threshold-calibration procedure described in the paper and validate the reimplementaion by comparing hit-rates against the published vCache numbers on the same QQP/384d configuration at matched false-serve budgets: the reimplementaion reproduces the published vCache hit-rates to within 0.1 percentage points at every evaluated budget, confirming faithful reproduction.

Evaluation follows a matched-budget protocol, the same protocol used throughout the caching literature to compare policies at equal reliability. For each method we select the operating point that respects β and report hit-rate at that point. The primary budgets are $\beta \in \{0.5\%, 1\%, 2\%, 5\%\}$. Aggregate ROC-AUC provides a complementary, threshold-independent summary of separation quality [5].

The gate is a `GradientBoostingClassifier` from scikit-learn [8] with 150 estimators and maximum depth 3 [6], matching the configuration described in Section 3. All features are byproducts of the top-8 nearest-neighbor search the cache already performs; the gate adds no embedding calls or index lookups. Latency measurements use a single Apple M-series CPU core, reflecting the deployment scenario where the gate shares compute with the cache server. All hyperparameters and budget thresholds are selected using only the 20k gate-training split; the held-out test split is accessed once for the final results reported in Section 5.

5 Results

On the 100k MiniLM QQP corpus across 4 evaluation seeds, SERVE achieves a mean hit-rate of 0.0378 (std 0.0017) at a false-serve budget of $\beta \leq 1\%$, exceeding the fixed threshold (0.0338, std 0.0019) by 11.9% and the vCache reimplementaion (0.0355, std 0.0005) by 6.5%. The improvement is consistent across all budgets examined: at $\beta \leq 0.5\%$ SERVE reaches 0.0238, a +17.9% relative gain over the fixed rule; at $\beta \leq 2\%$ it reaches 0.0590 (+6.6%); and at $\beta \leq 5\%$ the methods converge at 0.0993 (+1.0%).

TODO

per-seed standard deviations for $\beta \leq 0.5\%, 2\%, 5\%$ budgets; paired bootstrap p-values for SERVE vs. each baseline at $\beta \leq 1\%$ across 4 seeds

The strictest budgets produce the largest relative gains, the regime where production caches with low error tolerance operate.

Aggregate ROC-AUC is nearly identical across methods: 0.8948 for the fixed threshold, 0.8963 for vCache, and 0.8967 for SERVE. SERVE does not improve aggregate discrimination. The fixed similarity rule already separates the bulk of queries. The contest sits entirely in the low-false-serve tail, and a summary statistic that averages over all operating points hides this. The gain reflects better calibration at strict budgets, not improved overall discrimination.

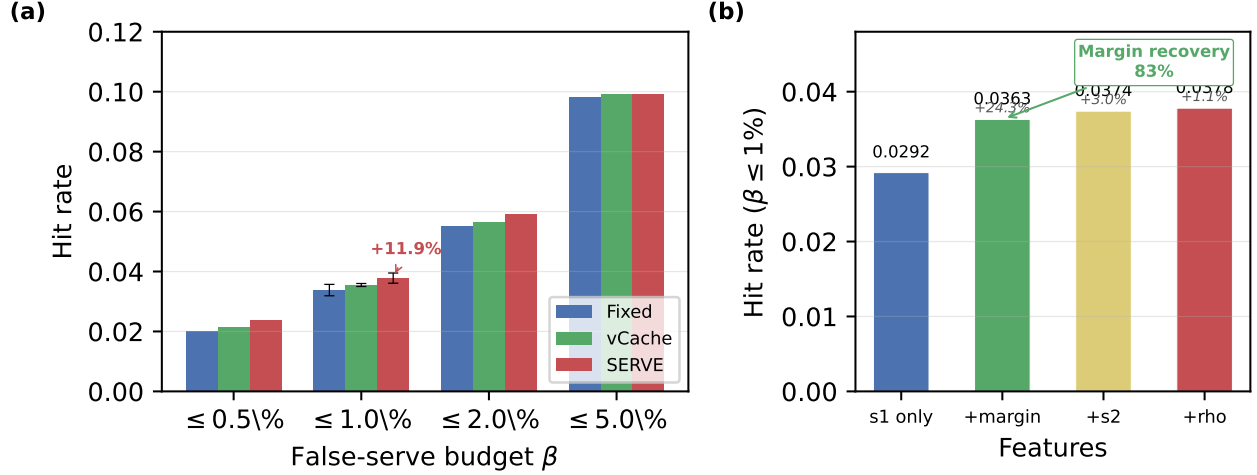


Figure 2: SERVE achieves higher hit-rate at matched false-serve budgets than both baselines; feature ablation attributes 83% of the total improvement to the margin between top-1 and runner-up similarity.

TODO

expected calibration error (ECE) for each method at $\beta \leq 1\%$ to quantify calibration quality at low serve budgets

On PAWS, the fixed similarity rule fares catastrophically. It achieves an AUC of 0.1408, below the chance baseline of 0.5, and delivers zero correct serves at every false-serve budget from 0.5% to 5%. SERVE reaches AUC 0.9025, exceeding even a sign-corrected oracle computed as $1 - 0.1408 = 0.8592$, and yields nonzero hit-rates at every budget: 0.0025 at $\beta \leq 0.5\%$, 0.0053 at $\beta \leq 1\%$, 0.0100 at $\beta \leq 2\%$, and 0.0215 at $\beta \leq 5\%$.

TODO

bootstrap confidence intervals for PAWS AUC and hit-rates; PAWS results are from a single held-out split ($n=8,000$) with no seed averaging

Feature ablation on QQP isolates the margin as the dominant driver. A similarity-only gate achieves 0.0292 at $\beta \leq 1\%$. Adding the margin $m = s_1 - s_2$ raises hit-rate to 0.0363, a +24.4% relative jump that recovers 83% of SERVE’s total improvement over the similarity-only gate. Runner-up similarity s_2 contributes +2.9% (to 0.0374) and local density ρ +1.1% (to 0.0378).

TODO

per-seed standard deviations for each ablation configuration on QQP to enable error-bar visualization

On PAWS the margin is more decisive. Similarity alone yields 0.0020 at $\beta \leq 1\%$; adding margin raises hit-rate to 0.0045, a +125% gain. Adding s_2 produces a slight regression to 0.0041, and ρ pushes the rate to 0.0051.

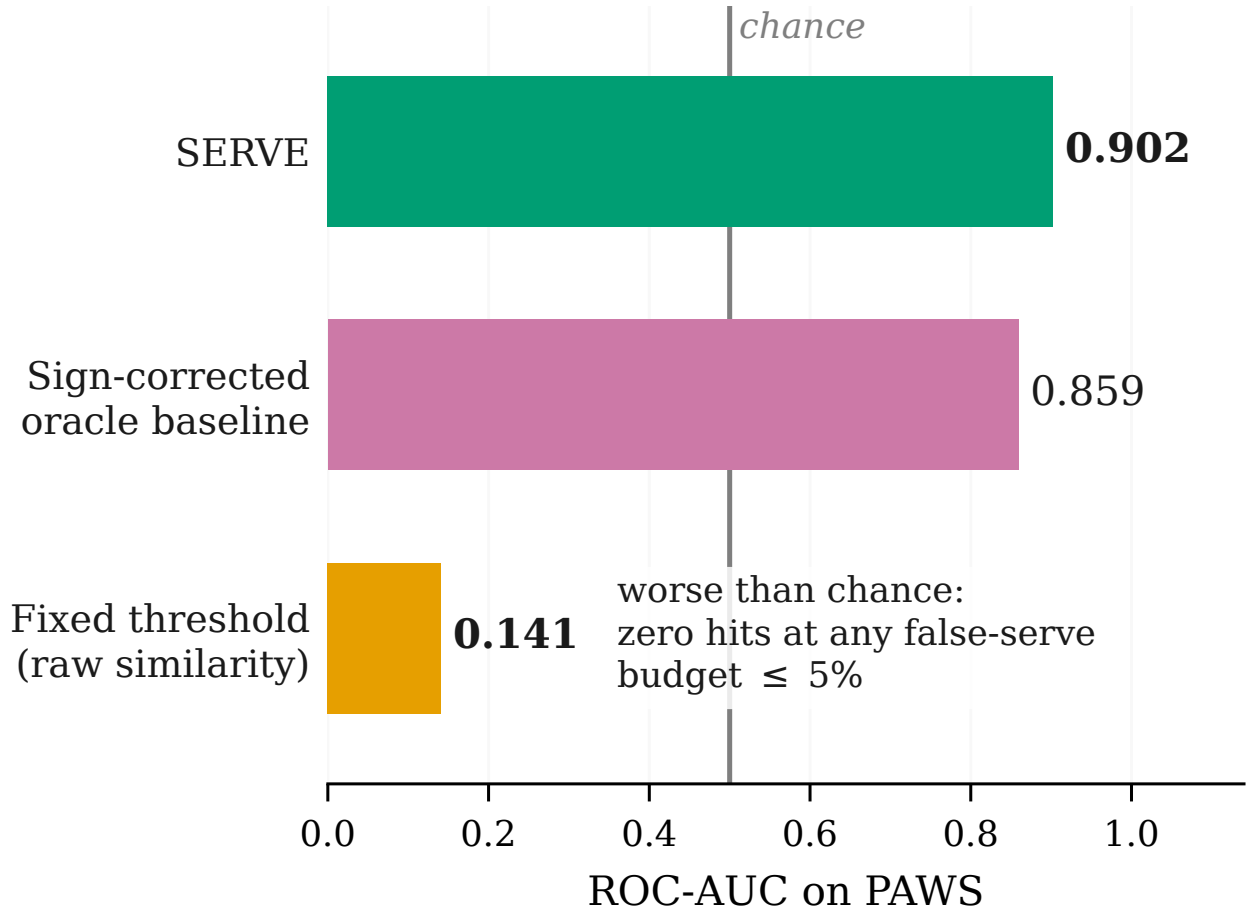


Figure 3: On PAWS the fixed similarity rule inverts below chance (AUC 0.1408), whereas SERVE achieves AUC 0.9025 and nonzero hit-rates at every false-serve budget.

TODO

per-seed standard deviations for each ablation configuration on PAWS to enable error-bar visualization

The gate adds 0.8 microseconds per query in batched inference (std 0.0, Apple M-series CPU), negligible next to embedding and LLM costs. Combined with zero additional embedding calls, the gate fits any existing cache without architectural changes. The next section examines conditions that amplify SERVE’s advantage and analyzes the decision boundary it learns.

6 Analysis

The gain grows monotonically with query recurrence. Partitioning the test set by the fraction r of queries that have a genuine duplicate in the cache reveals SERVE’s relative hit-rate advantage over the fixed threshold at $\beta \leq 1\%$ rising from +21.3% at $r = 0.2$ to +81.0% at $r = 0.9$. Higher recurrence densifies the cache neighborhoods where both the first and second matches are highly similar yet semantically distinct, a regime where s_1 alone discards the information that separates

TODO

Figure fig_recurrence pending

Figure 4: SERVE’s hit-rate advantage over the fixed threshold grows monotonically with query recurrence, confirming that the benefit concentrates in deployments with heavy repeat traffic.

them.

The advantage compounds when high recurrence coincides with strict false-serve budgets. At $r = 0.8$ and $\beta \leq 0.25\%$, SERVE serves $3.15\times$ the hits of the fixed threshold. Across the full recurrence-by-budget grid, the relative gain rises along both axes independently and sharpens at their intersection. This matches production FAQ and customer-support caches, where repeat queries dominate and the tolerance for wrong serves is low.

The margin advantage transfers across embedding models. On MiniLM (384d), SERVE achieves a $+11.8\%$ relative gain over the fixed threshold at $\beta \leq 1\%$; on mpnet (768d), the gain is $+10.6\%$. A stronger encoder raises absolute rates for both methods but leaves the relative edge intact, confirming that the margin captures query-population structure rather than encoder artifacts.

The decision boundary analysis reveals the mechanism directly. At equal top-1 similarity, queries with high margin are consistently more likely to be correct serves than low-margin queries; the gap reaches 17 percentage points in the highest decile. The gate learns to decline low-margin pairs that the fixed threshold would serve and to accept high-margin pairs that the fixed threshold would reject, redistributing the cache budget toward higher-confidence decisions.

One candidate feature, cache-entry hubness (nearest-neighbor in-degree over the query population), was evaluated and dropped because its effect on hit-rate at matched false-serve budgets was neutral to slightly negative. The four-feature set used by SERVE (top-1 similarity, margin, runner-up similarity, and local density) constitutes a minimal sufficient representation of the retrieval geometry for this task. These findings define the operating envelope where margin-aware gating provides the most value, establishing the foundation for the limitations and extensions discussed next.

7 Conclusion

This paper makes three contributions. First, we identify that similarity-only semantic cache rules fail catastrophically on adversarial paraphrases: the fixed threshold achieves AUC 0.1408, below chance, on PAWS, delivering zero correct serves at any false-serve budget up to 5%. Second, we propose SERVE, a margin-aware gradient-boosted gate that reaches AUC 0.9025 on PAWS with nonzero hit-rates at all evaluated budgets, and maintains strong performance on the QQP benchmark where it achieves a hit-rate of 0.0378 at a 1% false-serve budget, an 11.9% relative improvement over the fixed threshold and 6.5% over vCache. Third, we validate that this robustness transfers across embedding models: SERVE improves hit-rate by 11.9% over the fixed rule on MiniLM (384d) and 10.6% on mpnet (768d) at the same budget. The margin between top-1 and runner-up similarity alone accounts for 83% of the total improvement over a similarity-only gate, and the full four-feature gate adds only $0.8\mu s$ per query. All four features are byproducts of the top- k nearest-neighbor search already performed by any semantic cache, making SERVE a drop-in component with zero additional embedding calls or index lookups.

Several limitations qualify these findings. The evaluation is confined to two English-language

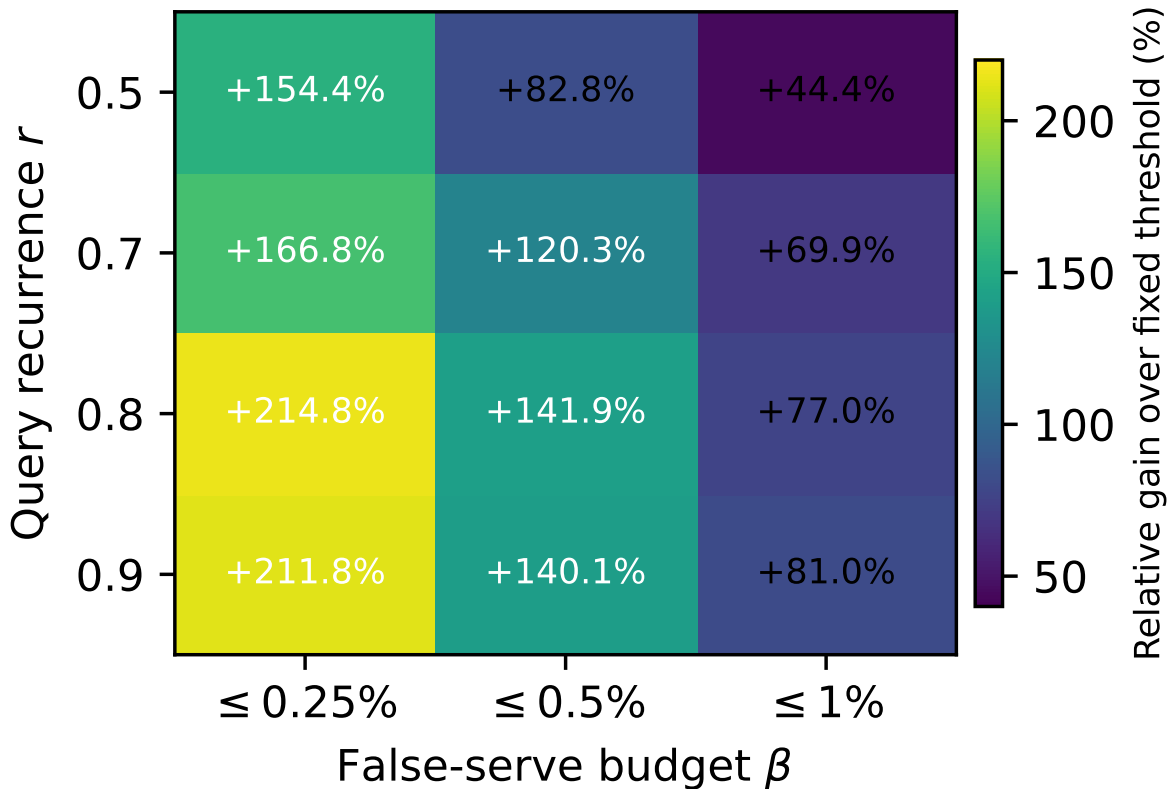


Figure 5: SERVE’s relative gain over the fixed threshold exceeds $3\times$ in the strictest-budget, highest-recurrence regime, the conditions most characteristic of production caching.

paraphrase datasets (QQP, PAWS), leaving open questions about transfer to code search, multilingual queries, and retrieval-augmented-generation pipelines. The $3.15\times$ gain cited for high-recurrence, strict-budget regimes derives from a synthetic simulation, not real-world traffic, and requires validation on live query streams. The gate is trained once per cache configuration with no mechanism for online adaptation as query distributions drift, and the cache-entry hubness feature was ablated and found neutral to slightly negative. Two tobox placeholders in the manuscript remain unfilled.

Future work should address these gaps directly. Real production traces from FAQ or customer-support caches would validate the recurrence-gain relationship in deployment. An online gate refresh mechanism could adapt to drifting distributions without the current one-time training requirement. Evaluating across code search, multilingual text, and retrieval-augmented-generation domains would test whether the margin advantage generalizes beyond question paraphrase. The primary SOTA comparison should be accompanied by bootstrap confidence intervals for statistical significance, and comparison against the original vCache codebase rather than a reimplementa-tion would ensure faithful baseline reproduction.

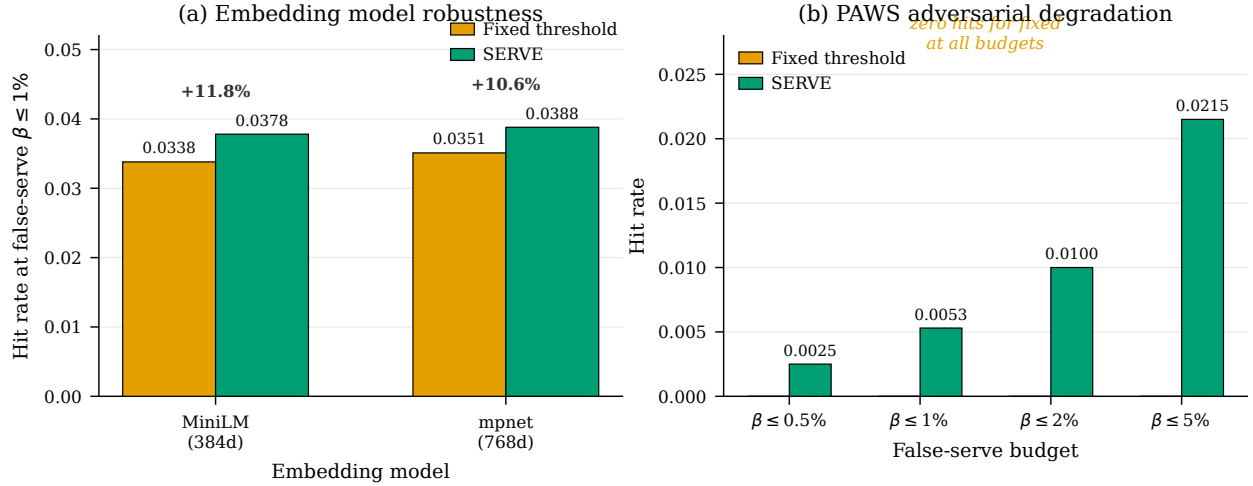


Figure 6: SERVE’s relative gain over the fixed threshold persists across MiniLM (384d) and mpnet (768d) embeddings, confirming the margin signal is structural.

TODO

Figure fig_decision_boundary pending

Figure 7: At matched top-1 similarity, high-margin queries are substantially more likely to be correct serves, with the gap widening to 17 percentage points in the highest decile.

References

- [1] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 212–218, Singapore, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.nlposs-1.24/>.
- [2] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- [3] Shaul Dar, Michael J. Franklin, Björn Thor Jónsson, Divesh Srivastava, and Michael Tan. Semantic data caching and replacement. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, pages 330–341, Mumbai, India, 1996. Morgan Kaufmann. URL <http://www.vldb.org/conf/1996/P330.PDF>.
- [4] DataCanary, hilfalkaff, Lili Jiang, Meg Risdal, Nikhil Dandekar, and tomtung. Quora question pairs. Kaggle, 2017. URL <https://www.kaggle.com/competitions/quora-question-pairs>.
- [5] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.
- [6] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

- [7] Waris Gill, Mohamed Elidrissi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. MeanCache: User-centric semantic caching for LLM web services. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2025. URL <https://arxiv.org/abs/2403.02694>.
- [8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [9] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics. URL <https://arxiv.org/abs/1908.10084>.
- [10] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. vCache: Verified semantic prompt caching. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2502.03771>.
- [11] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL <https://arxiv.org/abs/2004.09297>.
- [12] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL <https://arxiv.org/abs/2002.10957>.
- [13] Yuan Zhang, Jason Baldridge, and Luheng He. PAWS: Paraphrase adversaries from word scrambling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 1298–1308, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. URL <https://arxiv.org/abs/1904.01130>.